AAACET Virtual Assistant

Step-by-step guide: deploy, host, buy and connect the AI API, manage the vector database, and run it for AAA College of Engineering and Technology

What you will have at the end	
Public address	https://assistant.<your-college-domain> (for example assistant.aaaenggcoll.ac.in), with HTTPS
Answers	From the official website only, with links to every source; AI-written with a Claude API key
Search	Hybrid search (meaning + keywords) over 3,400 text and 4,800 image chunks in a Qdrant vector database
Extras	Suggestions while typing, questions in any language (with the key), embeddable on the college website
Running cost	A small cloud server (about US$12-25 / month) + Claude API usage (from under US$10 / month)

How to use this guide

Part A is a one-page quick path. Parts B-L explain every step. Part M lists things that are easy to forget, Part N fixes common problems, and Part O is the launch checklist to tick off.

Commands in grey boxes are typed on the **server** (Linux) unless marked otherwise. Replace values in <angle brackets> with your own.

Prepared for the RAG-Sys project (branch claude/admiring-johnson-xhrq59). Tested on the same Docker setup: image build, data loading, indexing into Qdrant, chat, suggestions, per-visitor rate limiting, admin access and HTTPS through Caddy (local certificate).

A. The quick path (one page)

1. **Get approval** from the principal / IT in-charge: the address, who maintains it, the monthly budget (Part B).
2. **Rent a Linux server**: Ubuntu 24.04, 2 vCPU, 4 GB RAM (8 GB is faster for the first indexing), 40 GB disk, in an Indian region (Part C).
3. **Point a subdomain** such as assistant.aaaenggcoll.ac.in to the server's IP address (Part D).
4. **Install Docker**, copy the project to the server, fill in `.env` (Part E).
5. **Start it**: build, load the knowledge base, switch on HTTPS (Part F).
6. **Buy Claude API credits**, create a key, put it in `.env`, restart (Part G).
7. **Test** with the checklist (Part F.6), then **add it to the college website** (Part J).
8. **Schedule a weekly refresh** from the live website and **back up** (Parts I and K).

```
# On the server, after Parts C-E (Ubuntu 24.04 with Docker installed):
cd /opt/aaacet/app
cp .env.example .env && nano .env                  # API key, admin token, domain (Part E.3)
docker compose up -d --build                        # build and start (5-15 min the first time)
docker compose run --rm --no-deps --user root -v "$PWD/data/processed:/seed:ro" backend \
  sh -c "mkdir -p /data/processed && cp -r /seed/. /data/processed/ \
         && chown -R app:app /data/processed"
docker compose restart backend                      # indexes the data; chat works meanwhile
docker compose --profile https up -d                # HTTPS on your domain (free certificate)
```

What is already done for you

The website has been crawled (2,115 pages, 4,785 images) and cleaned into `data/processed`, which ships with the project. You do **not** need to crawl before launch; the server only has to index it into the vector database, which happens automatically on start.

B. Before you start: decisions and approvals

A college-facing service needs a few non-technical decisions first. Getting them settled avoids rework later.

Decision	Recommendation	Who decides
Public address	A subdomain of the college domain, e.g. **assistant.aaaenggcoll.ac.in**. Free; needs one DNS record from whoever manages the college domain.	Principal / IT / website vendor
Where it runs	A small cloud server in India (simplest), or a college server if IT can open ports 80/443 to the internet (Part C).	IT in-charge
AI answers	Buy Claude API credits with a college account and card; start with a small amount and a spend limit (Part G). Without a key it still works, quoting the website.	Accounts / management
Owner	Name one staff member responsible for the server, the API account, updates and wrong-answer reports.	HOD / principal
Content	The assistant only knows what is on the official website. Wrong or old information must be fixed **on the website**, then refreshed (Part I).	Website / office staff
Privacy notice	Tell users not to type personal details, that questions are processed by an AI service, and to confirm important details (fees, dates) with the office.	Management

Branch and code

The finished work is on the GitHub branch `claude/admiring-johnson-xhrq59` of `ArunKumar071106/RAG-Sys`. Merging it into `main` through a pull request before deploying keeps one clear version to deploy and update.

C. Choose where to host

The app is four Docker containers (web page, backend, vector database, HTTPS). It needs a machine that runs continuously with Docker. Shared web hosting (cPanel / PHP hosting) **cannot** run it, and static hosts such as GitHub Pages or Netlify can only serve the page, not the backend.

Option	Good for	Approx. cost	Notes
Cloud VM in India (recommended): DigitalOcean (Bengaluru), AWS Lightsail/EC2 (Mumbai, Hyderabad), Azure (Pune, Chennai), Google Cloud (Mumbai, Delhi)	Reliable public service with little IT effort	US$12-25 / month for 2 vCPU, 4 GB	Pick Ubuntu 24.04 LTS, x86-64. Easiest to get HTTPS working.
Oracle Cloud free tier (Mumbai, Hyderabad)	Zero hosting cost	Free (Arm VMs)	Arm processors: the images should build, but this setup was not tested on Arm. Free capacity is not always available.
College's own server	No monthly bill; data stays on campus	Hardware you have	Needs a public IP or port forwarding of 80/443 from the college router/firewall, and someone to keep it running.

Option	Good for	Approx. cost	Notes
Your laptop / PC	Demos only	-	Turns off with the PC; not for students.

Server size

Resource	Minimum	Recommended	Why
CPU	2 vCPU	2-4 vCPU	Indexing the knowledge base and answering run on the CPU
Memory	4 GB	8 GB	Backend + embedding model + Qdrant; 8 GB makes the first indexing faster
Disk	30 GB	40-60 GB SSD	Docker images (~1.5 GB), data, model, logs, room for a full re-crawl with images
OS	Ubuntu 22.04	Ubuntu 24.04 LTS	Commands in this guide are for Ubuntu

A 2 vCPU / 4 GB server comfortably serves a college (dozens of people chatting at the same time). The slow part, the first build of the vector index, happens once and the chat works while it runs.

D. Domain name and DNS

1. Create the server first (Part E.1) and note its **public IPv4 address**, e.g. 203.0.113.25.
2. Ask the person who manages **aaaenggcoll.ac.in** (IT team, website vendor or domain registrar account) to add:

Type	Name / Host	Value	TTL
A	assistant	<server public IP>	300 (5 min) or default

1. Wait 5-30 minutes, then check from your PC: `nslookup assistant.aaaenggcoll.ac.in` must show the server's IP. HTTPS (Part F.5) only works after this.

No college domain access yet?

You can start with the server's IP address over plain HTTP (http://<server-ip>:8080) for internal testing. Do not announce it to students without HTTPS. A separate domain (e.g. a .in name, roughly Rs.500-1,000 a year) also works, but a subdomain of the official domain is more trustworthy for students.

E. Set up the server

E.1 Create the server

1. Sign up with the cloud provider (college account). Create a VM / Droplet / instance: **Ubuntu 24.04 LTS**, 2 vCPU / 4-8 GB, 40 GB SSD, region **India**.
2. Authentication: choose **SSH key** (safer than a password). On Windows, PowerShell can create one: `ssh-keygen -t ed25519`, then paste the contents of `~/.ssh/id_ed25519.pub`.
3. In the provider's firewall / security group allow inbound **22** (SSH), **80** and **443** (web). Nothing else.
4. Connect from your PC: `ssh ubuntu@<server-ip>` (the user may be root, ubuntu or azureuser depending on the provider).

E.2 Install Docker and basic protection

```
sudo apt update && sudo apt upgrade -y
curl -fsSL https://get.docker.com | sudo sh          # official Docker install script
sudo usermod -aG docker $USER                        # docker without sudo (after re-login)
sudo apt install -y git unattended-upgrades          # git + automatic security updates
sudo ufw allow OpenSSH && sudo ufw allow 80,443/tcp && sudo ufw enable
```

Log out (`exit`) and connect again, then check: `docker --version` and `docker compose version`.

E.3 Copy the project to the server

Option 1 - git (best for updates). The repository is private, so GitHub asks for your username and a **personal access token** (GitHub, Settings - Developer settings - Fine-grained tokens, read-only access to this repository) instead of a password.

```
sudo mkdir -p /opt/aaacet && sudo chown $USER /opt/aaacet && cd /opt/aaacet
git clone -b claude/admiring-johnson-xhrq59 https://github.com/ArunKumar071106/RAG-Sys.git app
cd app
```

Option 2 - ZIP. Download the branch ZIP on your PC, then upload and unpack it (from PowerShell on your PC):

```
scp RAG-Sys.zip ubuntu@<server-ip>:/opt/aaacet/        # on your PC
ssh ubuntu@<server-ip>
cd /opt/aaacet && sudo apt install -y unzip && unzip RAG-Sys.zip && mv RAG-Sys-* app && cd app
```

E.4 Fill in the settings file (.env)

```
cp .env.example .env
openssl rand -base64 32          # copy the output: this is your ADMIN_TOKEN
nano .env                        # edit, then Ctrl+O, Enter, Ctrl+X to save and exit
```

Setting in .env	Set it to	Why
ANTHROPIC_API_KEY	Your key from Part G (sk-ant-...); can be added later	AI-written answers, any language
ADMIN_TOKEN	The random string from openssl	Protects re-indexing (Part I). Never share it.
LLM_MODEL	claude-sonnet-5-5 (balanced) or claude-haiku-5-5 (cheapest)	Cost / quality choice (Part G.4)
DOMAIN	assistant.aaaenggcoll.ac.in (remove the # in front)	Caddy gets the HTTPS certificate for it

Setting in .env	Set it to	Why
FRONTEND_PUBLISH	127.0.0.1:8080 (remove the #)	Plain-HTTP port only reachable on the server itself
COMPOSE_PROFILES	https (add this new line)	So every 'docker compose up' also starts Caddy
RATE_LIMIT_PER_MINUTE	20 (default 30)	Questions per visitor per minute; protects your API budget

Keep .env secret

It holds the API key and admin token. Never put it in GitHub, WhatsApp or e-mail. Keep a copy in a password manager or with the principal's office (Part K).

F. Start the app

F.1 Build and start the containers

```
cd /opt/aaacet/app
docker compose up -d --build       # first build: 5-15 minutes (downloads Python packages)
docker compose ps                  # qdrant, backend, frontend (and caddy) should be "running"
```

F.2 Load the knowledge base into the server

The cleaned website content ships in `data/processed`. Copy it into the backend's data volume (this exact command was tested), then restart the backend:

```
docker compose run --rm --no-deps --user root -v "$PWD/data/processed:/seed:ro" backend \
  sh -c "mkdir -p /data/processed && cp -r /seed/. /data/processed/ \
         && chown -R app:app /data/processed"
docker compose restart backend
```

F.3 Watch it build the vector index

```
docker compose logs -f backend     # Ctrl+C stops watching (the app keeps running)
```

You will see, in order: `Answering with keyword search while the semantic index is built` (chat works from here), progress lines `Indexing aaacet_text: 1152/3426 (34%)` and `aaacet_images`, then `Semantic index ready; answering with hybrid search`. On a 2-4 vCPU server expect roughly 10-40 minutes, once. The first start also downloads the 67 MB embedding model.

F.4 Check health

```
curl -s http://127.0.0.1:8080/api/health
# {"status":"ok", ..., "components":{"llm":"anthropic:claude-sonnet-5-5", ...},
#  "search":"hybrid"}
```

F.5 Switch on HTTPS

With DNS pointing at the server (Part D) and DOMAIN set:

```
docker compose --profile https up -d      # not needed if COMPOSE_PROFILES=https is in .env
docker compose logs caddy | grep -i certificate
```

Caddy obtains a free Let's Encrypt certificate and renews it automatically. Open **https://assistant.aaaenggcoll.ac.in**; plain http:// redirects to https://.

F.6 Test before announcing

- ☐ The page loads on a phone and a laptop over https:// with the padlock.
- ☐ Typing 'ai&ds fac' shows a suggestion; picking it lists the AI & DS faculty table.
- ☐ Try: intake for B.E. and B.Tech, hostel facilities, bus transport, contact phone and e-mail, photos of the library.
- ☐ Each answer has source links that open the official website.
- ☐ With the API key: answers are written in plain words; a question in Tamil is answered in Tamil.
- ☐ An unrelated question (e.g. 'weather on the moon') says it is not covered instead of guessing.
- ☐ Ask the same question 25 times quickly: after the rate limit you see 'sending messages too quickly'.

G. Buy the Claude API and connect it

The app calls Anthropic's **Claude API** (official `anthropic` Python library) to write answers and to translate non-English questions. Search, embeddings and OCR run free on your server.

G.1 Create the account and buy credits

1. Go to **console.anthropic.com** and sign up with an official college e-mail (so the account belongs to the college, not a person). Create an organization named after the college.
2. Open **Settings - Billing**. Add a payment method: a card enabled for **international online payments** (many Indian debit cards need this switched on in the bank app), or the college's corporate card.
3. Buy a small amount of **prepaid credits** to start (for example US$10-20). Usage is deducted as students ask questions.
4. Open **Settings - Limits** (or the billing page) and set a **monthly spend limit** and e-mail alerts, so a busy week cannot overspend.
5. Open **API Keys** - **Create Key**, name it `aaacet-assistant-production`, copy it immediately (it is shown once). It starts with `sk-ant-`.

Screens and names in the Console change from time to time; if a menu differs, look for Billing, Limits and API Keys.

G.2 Connect the key to the app

```
cd /opt/aaacet/app
nano .env              # set ANTHROPIC_API_KEY=sk-ant-...  and  LLM_MODEL=claude-sonnet-5-5
docker compose up -d --force-recreate backend
curl -s http://127.0.0.1:8080/api/health
# "llm":"anthropic:claude-sonnet-5-5" in the output means the key is connected
```

If it still says `extractive`, check for spaces or quotes around the key, then `docker compose logs backend | grep -i anthropic`.

G.3 What the API does in this app

Job	Setting	Notes
Write answers from the retrieved website text, with citations	LLM_MODEL, LLM_EFFORT=low	Only official content is sent; links not from the website are removed
Translate a non-English question into an English search	same model	Answer comes back in the user's language
Describe images (optional)	VISION_ENABLED=true, VISION_MODEL	Off by default; only needed for a re-crawl
Back-up model if Claude declines a request	LLM_SERVER_FALLBACKS=true	Opus and Sonnet only; Haiku has no fallback

G.4 Which model, and what it costs

Model	LLM_MODEL	Price per million tokens (in / out)	Per question*	9,000 questions / month*
Claude Haiku 5.5	claude-haiku-5-5	US$0.10 / US$0.50	< US$0.001	about US$8
Claude Sonnet 5.5 (recommended start)	claude-sonnet-5-5	US$2 / US$10	about US$0.02	about US$160
Claude Opus 5.5 (code default)	claude-opus-5-5	US$4 / US$20	about US$0.04	about US$320

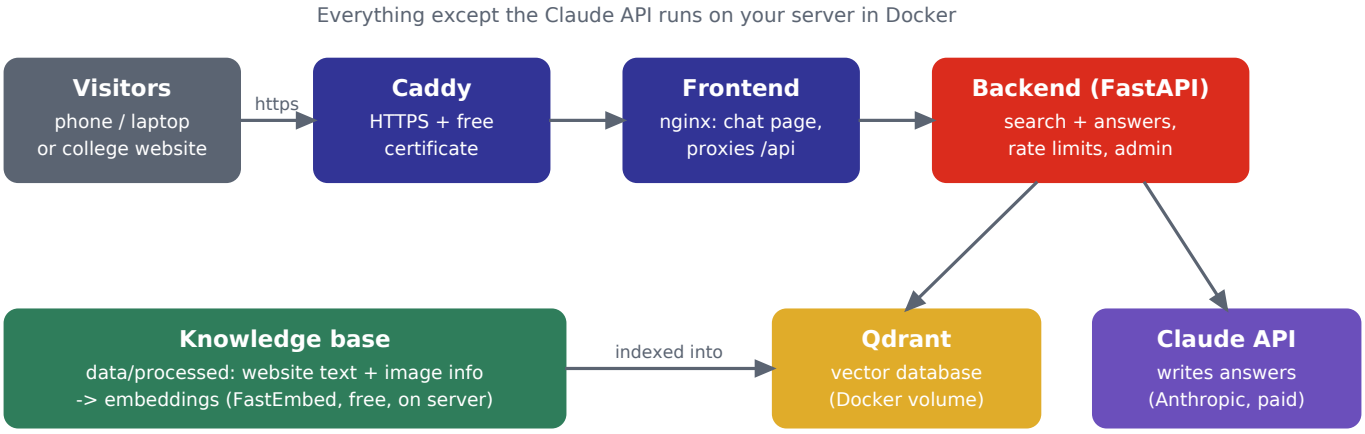
* Estimates: about 6,000 tokens of website text in and 600 out per answer (300 questions a day). Real use varies; watch the Console's usage page during the first weeks. Prices are Anthropic's list prices at the time of writing.

How to choose

Ask 15 typical questions (faculty, intake, hostel, transport, one in Tamil) with **claude-haiku-5-5**, then the same with **claude-sonnet-5-5**. If Haiku's answers are just as clear and correct, keep Haiku: it is about 20 times cheaper. Change the model any time in .env and recreate the backend (G.2).

H. The vector database: what it is, which is best, how your data gets in

H.1 How your website becomes searchable



Stage	What happens	Where / tool
1. Crawl	Downloads the official website pages, PDFs and image addresses (polite: obeys robots.txt, 1 s delay)	Scrapy
2. Extract	Cleans menus and footers, keeps headings, tables (labelled rows) and image context	BeautifulSoup
3. Images	Downloads images, removes duplicates, reads text in images (OCR)	RapidOCR (free, local)
4. Chunk	Splits pages into pieces of up to ~800 tokens along headings; tables keep their header	data/processed/*.jsonl
5. Embed + index	Turns every chunk into a vector (meaning) and a keyword index	FastEmbed BGE-small -> **Qdrant**; BM25
6. Answer	Hybrid search finds the best chunks; Claude writes the answer with citations	Backend + Claude API

Stages 1-4 are already done and shipped. Stage 5 runs automatically when the backend starts (Part F.2-F.3).

H.2 Which vector database is best for this?

Database	Type	Cost	Fit for this app
Qdrant (used, recommended)	Open source, runs in Docker on your server	Free	Already integrated and tested; fast; ~8,000 vectors is tiny for it; no extra account
Qdrant Cloud	Managed Qdrant	Free 1 GB cluster, then paid	Only if you do not want the database on your server: set QDRANT_URL and QDRANT_API_KEY in .env
pgvector (PostgreSQL)	Extension for Postgres	Free	Good if the college already runs Postgres; would need code changes here
Pinecone	Managed service	Paid beyond a small free tier	Works, but adds a vendor and cost for no gain at this size
Chroma, Weaviate, Milvus	Open source	Free	Fine products; no advantage over Qdrant here and would need code changes

Recommendation: keep Qdrant in Docker (the default). The whole college knowledge base takes only tens of megabytes, so the database is not a cost or speed concern.

H.3 Which embedding model?

EMBEDDING_PROVIDER / model	Speed on a CPU	Quality	When to use
fastembed + BAAI/bge-small-en-v1.5 (default)	Fast (minutes)	Good	Recommended start; 67 MB download
fastembed + BAAI/bge-base-en-v1.5	~3x slower to index	A little better	If you have 4+ vCPU and want slightly better search
bge-m3 (needs WITH_ML: "true" in docker-compose.yml)	Much slower, +2 GB image	Best, multilingual	Many non-English questions **without** a Claude key (with a key, questions are translated anyway)

Changing the model rebuilds the vector index automatically on the next start (the old vectors do not fit the new model).

H.4 Add or change content

- **Fix or add information on the official website** (including PDFs linked from it), then refresh (Part I). This keeps one source of truth: the assistant never contradicts the website.
- **Specific new pages** only: list their URLs in a text file and run `docker compose run --rm backend python -m rag.cli crawl --urls /data/new_urls.txt`, then refresh without re-crawling everything.
- Pages outside `aaaenggcoll.ac.in` are ignored on purpose (CRAWL_ALLOWED_DOMAINS).

I. Keep the knowledge base up to date

The website changes (new notices, faculty, events). Refresh the assistant **weekly**; it re-reads the site, re-embeds only changed pages and swaps in the new index without downtime.

I.1 Refresh now

```
curl -s -X POST http://127.0.0.1:8080/api/admin/reindex \
  -H "Authorization: Bearer <ADMIN_TOKEN>" -H "Content-Type: application/json" \
  -d '{"recrawl": true}'
# -> {"id":"5f2c...","state":"queued",...}
# check progress with the id from the answer:
curl -s http://127.0.0.1:8080/api/admin/jobs/<id> -H "Authorization: Bearer <ADMIN_TOKEN>"
```

A full re-crawl of 2,000+ pages and 4,800 images takes several hours because the crawler is deliberately polite. The assistant keeps answering with the current index meanwhile.

I.2 Refresh automatically every Sunday at 2 a.m.

First save the refresh command as a small script (paste your real token in place of <ADMIN_TOKEN>):

```
cat > $HOME/aaacet-refresh.sh <<'SH'
#!/bin/sh
curl -s -X POST http://127.0.0.1:8080/api/admin/reindex \
  -H "Authorization: Bearer <ADMIN_TOKEN>" -H "Content-Type: application/json" \
  -d '{"recrawl": true}'
echo " $(date)"
SH
chmod 700 $HOME/aaacet-refresh.sh                  # only you can read it (it holds the token)
$HOME/aaacet-refresh.sh                            # try it once by hand
```

Then schedule it:

```
crontab -e                                         # choose nano, add this one line, save:
0 2 * * 0 $HOME/aaacet-refresh.sh >> $HOME/aaacet-refresh.log 2>&1
```

> **After urgent website changes**
>
> Run I.1 by hand (e.g. after admission notices are published) instead of waiting for Sunday.

J. Put it on the college website

Ask the website team to add a link in the menu (e.g. 'Ask AAACET') and/or embed the chat on a page:

```
<iframe src="https://assistant.aaaenggcoll.ac.in/?embed=1" title="AAACET Virtual Assistant"
        style="width:100%;height:640px;border:0" loading="lazy"></iframe>
```

- ?embed=1 shows only the chat panel, without the header and footer.
- Embedding is allowed from https://aaaenggcoll.ac.in and https://www.aaaenggcoll.ac.in. For another site, add it to the `frame-ancestors` line in `frontend/nginx.conf` and rebuild (`docker compose up -d --build frontend`).
- WordPress: add a Custom HTML block with the iframe code on a page such as 'Virtual Assistant'.

K. Updates and backups

K.1 Install a new version of the app

```
cd /opt/aaacet/app
git pull                                  # or upload and unpack the new ZIP over the folder
docker compose up -d --build              # rebuilds only what changed; the data volume is kept
```

If an update ships a newer `data/processed` and you do **not** run your own refreshes, repeat F.2 to load it; the backend re-indexes only the changed pages.

K.2 What to back up

Item	How	Why
.env	Copy into a password manager / sealed office file	Contains the API key and admin token
Data volume (rag_data)	Weekly, command below; keep a copy off the server	Crawled content and index (can be rebuilt, but takes hours)
Qdrant volume	Optional	Rebuilt automatically from rag_data on start
Certificates (caddy_data)	Optional	Caddy re-issues certificates automatically

```
docker volume ls | grep rag_data                      # e.g. app_rag_data
docker run --rm -v app_rag_data:/data -v $HOME:/backup alpine \
  tar czf /backup/rag_data-$(date +%F).tgz -C /data .
```

Most cloud providers also offer automatic weekly **server snapshots** for a small fee; switching them on is the simplest full backup.

L. Security and monitoring

L.1 Security checklist

- ☐ SSH with keys only; disable password login (PasswordAuthentication no in /etc/ssh/sshd_config).
- ☐ Firewall allows only 22, 80, 443. FRONTEND_PUBLISH=127.0.0.1:8080 keeps the plain-HTTP port private. Note: ports published by Docker bypass ufw, so keep this setting.
- ☐ Strong random ADMIN_TOKEN; never in screenshots or chats. The backend and Qdrant are never published to the internet.
- ☐ API spend limit and alerts set in the Anthropic Console; rate limit per visitor set in .env (20-30 per minute).
- ☐ Automatic security updates on (unattended-upgrades) and a monthly 'sudo apt upgrade -y && sudo reboot'.
- ☐ Rotate the API key if it may have leaked (create a new key, update .env, recreate backend, delete the old key).
- ☐ Privacy notice on the page: no personal details, AI-processed, confirm important information with the office. Review Anthropic's commercial terms and data policy with the college (India's DPDP Act 2023 applies to personal data).

L.2 Daily and weekly checks

Check	Command / tool	Healthy result
Containers running	docker compose ps	All 'running'; restart: unless-stopped brings them back after a reboot
Health	curl -s https://assistant.../api/health	"status":"ok", "search":"hybrid"
Errors	docker compose logs --since 24h backend | grep -i error	Nothing new
Uptime alerts	UptimeRobot / Better Stack (free plans): monitor /api/health every 5 min	E-mail if it goes down
Disk space	df -h /	Below 80% used
API cost	console.anthropic.com usage page	Within the monthly budget

M. Things that are easy to miss

- **Accuracy disclaimer**: the page footer already says answers come from the website; keep fees and dates confirmed by the office, especially during admissions.
- **Wrong answers**: give students a way to report them (an e-mail or Google Form link). Most fixes are website fixes followed by a refresh.
- **Admission season load**: before counselling, check the API spend limit and consider a 4 vCPU server for that month.
- **Ownership**: the API account, server account and domain should be in the college's name, with two people having access.
- **Handover notes**: keep this guide, the server IP, account names (not passwords) and the refresh schedule in the IT file.
- **Accessibility**: the chat works with the keyboard and screen readers; keep it that way when the website team embeds it.
- **Robots and permission**: the crawler only reads the college's own public website; get written approval from the website owner/vendor if they host it.

- **Logs and privacy**: logs record requests, not answers; do not add analytics that store students' questions without a privacy notice.
- **Costs in rupees**: card payments in US$ may add bank currency-conversion charges and taxes; plan the budget with a margin.
- **Branch**: after merging the branch into main, deploy from main so updates are simple (git pull).

N. Troubleshooting

You see	Fix
Browser cannot open https://assistant...	DNS not pointing to the server yet (nslookup), ports 80/443 closed in the provider firewall, or Caddy not started: docker compose --profile https up -d; docker compose logs caddy
Chat: 'still starting up'	The backend is loading; wait a minute. docker compose logs backend shows progress.
Chat: 'Something went wrong'	docker compose ps; docker compose logs backend | tail -50
Answers quote the website instead of AI text	ANTHROPIC_API_KEY missing/invalid or no credits: check .env, recreate backend (G.2), check Console billing
'sending messages too quickly' for everyone	Visitors must reach the app through Caddy/nginx (not the backend directly); raise RATE_LIMIT_PER_MINUTE if many students share one campus IP address
PermissionError on /data/processed	Load the data with the exact F.2 command (copy as root, then chown to app)
'Collection has N-d vectors but the embedder produces M-d'	You changed the embedding model with an index from another model; it rebuilds on start, or run docker compose run --rm backend python -m rag.cli index --full
Out of disk space	docker system prune -f (removes old images); enlarge the disk
Port 80/443 already in use	Another web server (Apache/nginx) is installed on the host: stop and disable it

O. Launch checklist

Before launch

- ☐ Approval from management; owner named; privacy notice wording agreed.
- ☐ Server created (Ubuntu 24.04, 2 vCPU, 4-8 GB, India region), SSH key login, firewall 22/80/443.
- ☐ Docker installed; project copied to /opt/aaacet/app.
- ☐ .env filled: ANTHROPIC_API_KEY, ADMIN_TOKEN, LLM_MODEL, DOMAIN, FRONTEND_PUBLISH, COMPOSE_PROFILES=https.
- ☐ DNS A record for the subdomain points to the server (nslookup shows the IP).
- ☐ docker compose up -d --build; knowledge base loaded (F.2); logs show 'Semantic index ready'.
- ☐ https:// works with a padlock; http:// redirects.
- ☐ Claude credits bought, spend limit and alerts set, /api/health shows the anthropic model.
- ☐ Test checklist F.6 passed on phone and laptop.

After launch

- ☐ Link / embed added to the college website (Part J).
- ☐ Weekly refresh scheduled with cron (Part I.2); first refresh run checked.
- ☐ Backups: .env stored safely; weekly data-volume backup or provider snapshots on (Part K).
- ☐ Uptime monitor on /api/health with e-mail alerts.
- ☐ First month: review API usage weekly; adjust LLM_MODEL and limits.
- ☐ Wrong-answer reports reviewed monthly; website corrected and refreshed.

Need to start over?

Everything lives in /opt/aaacet/app and three Docker volumes. 'docker compose down' stops the app and keeps the data; the knowledge base can always be reloaded from data/processed (F.2).